

AF Seminar 5

1. Complexitate algoritmi
2. Numarul de arbori de distante minime
3. Arbori de distante care să conțină / nu conțină
o muchie
4. Cel mai lung drum în DAG
5. Nr. de arbori parțiali în graf

ex 1

Examen 2024 - 2025

6 graf orientat

n noduri (multiplu de 3)

$$m = \frac{n(n-1)}{3}$$

+

ce algoritm este mai rapid?

a) Floyd Warshall

b) Dijkstra (coadă de prioritate) din fiecare vârf

c) Dijkstra (vector) pt fiecare vârf

d) Bellman Ford pt fiecare vârf

+

a) $O(N^3)$ "pt fiecare vârf"

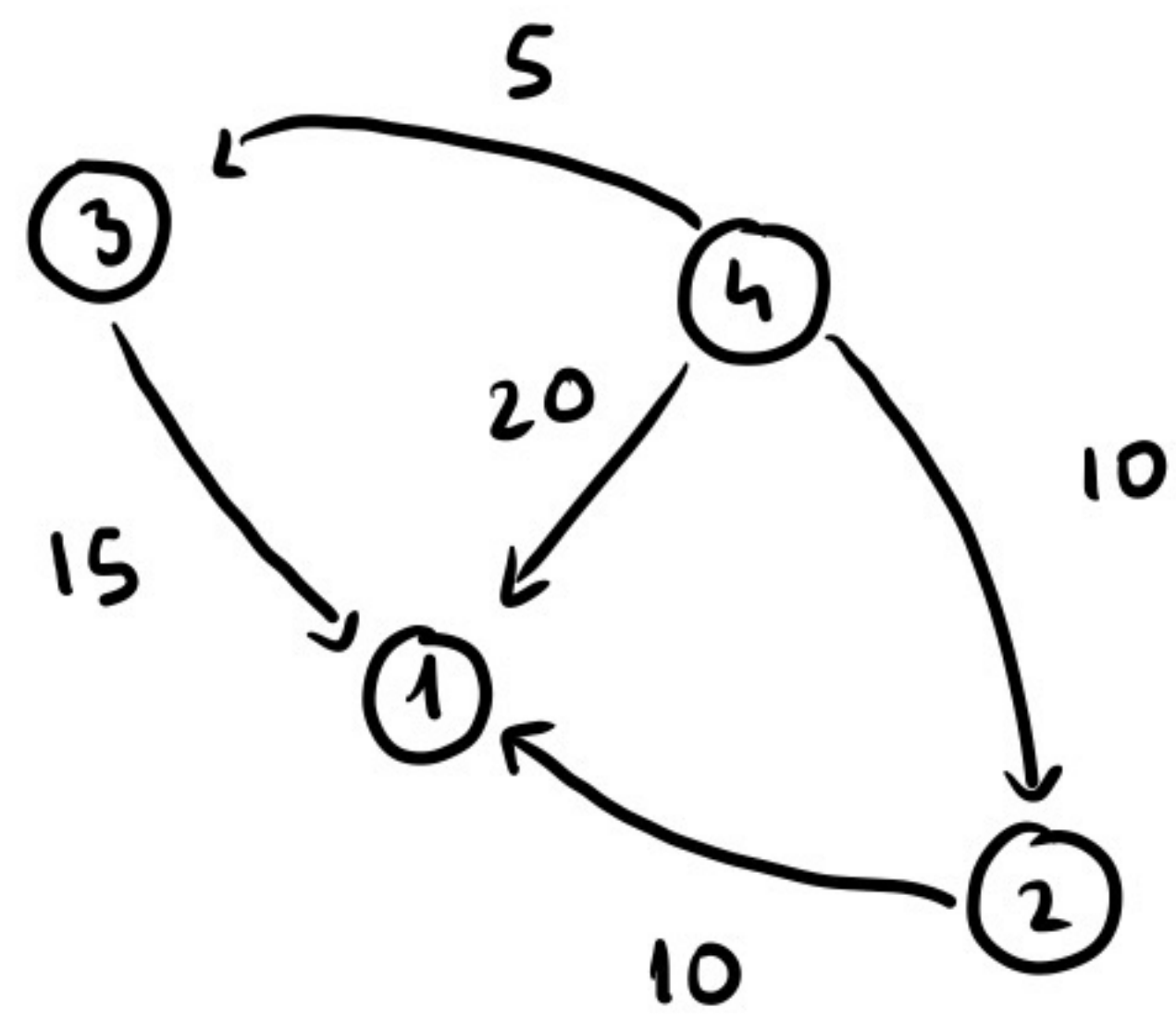
b) $O(N \cdot N \log M) = O(N^2 \log N)$

c) $O(N^3) = O(N \cdot N^2)$

d) $O(N \cdot N \cdot M) = O(N^2 \cdot M) = O(N^4)$



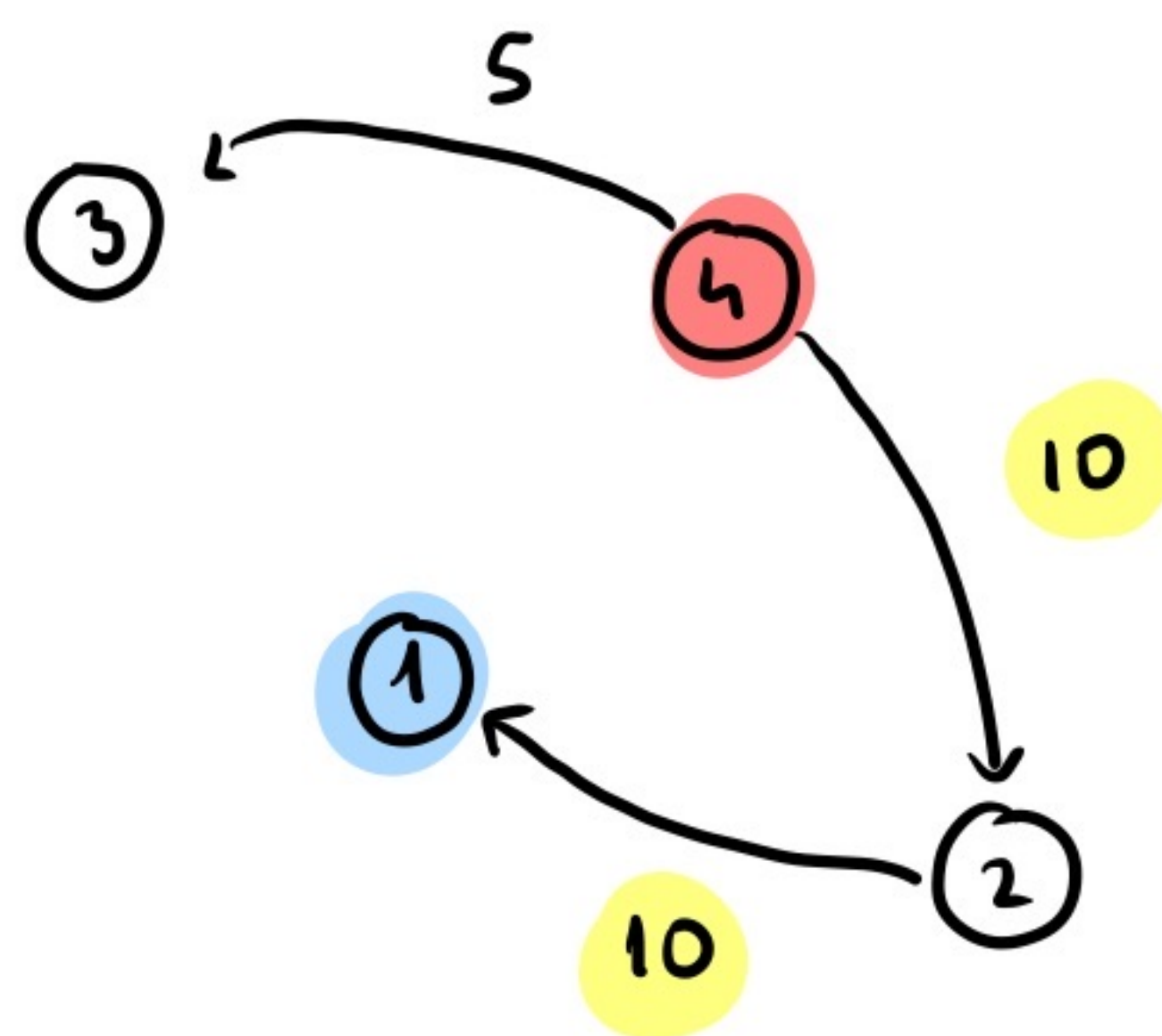
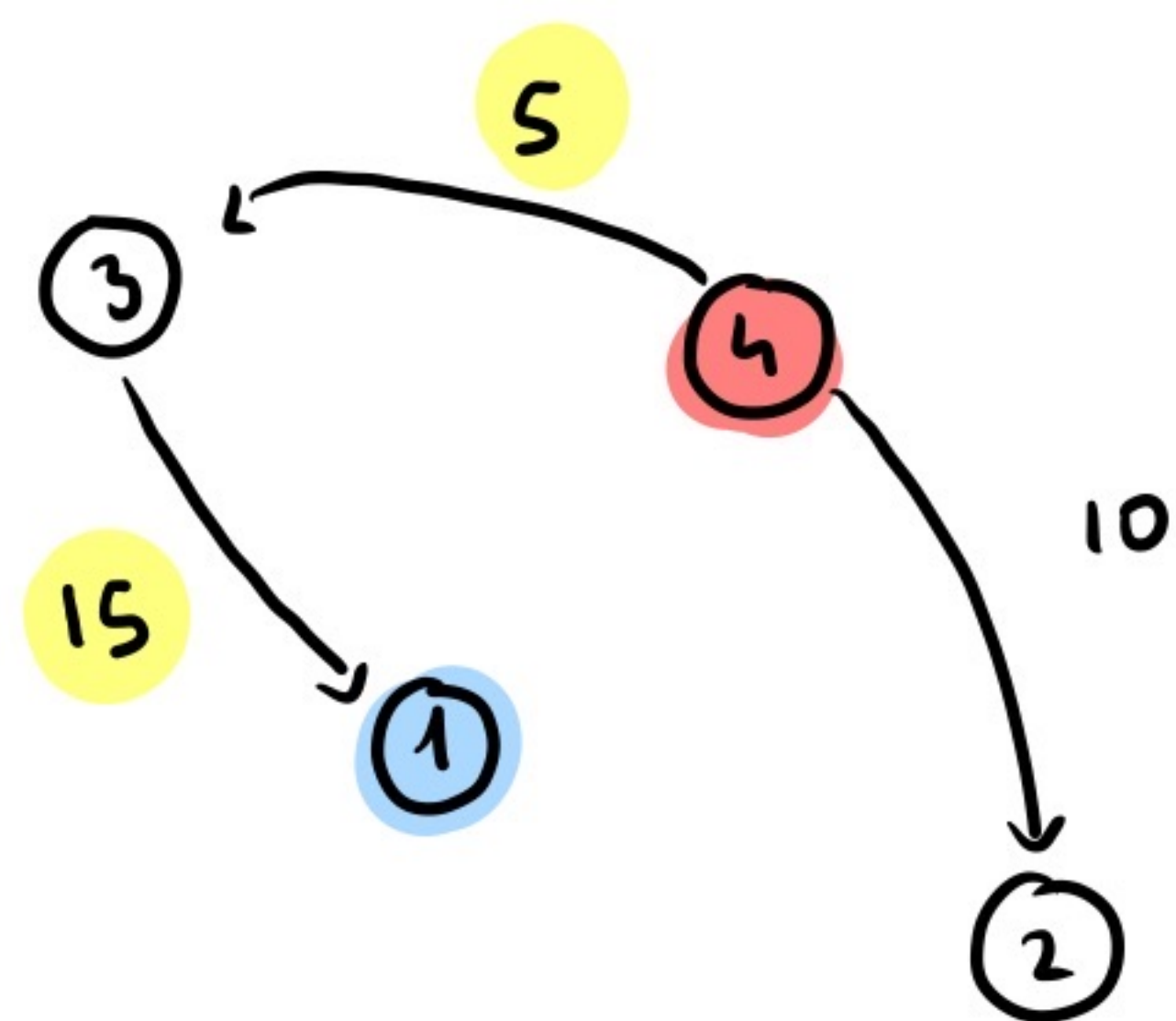
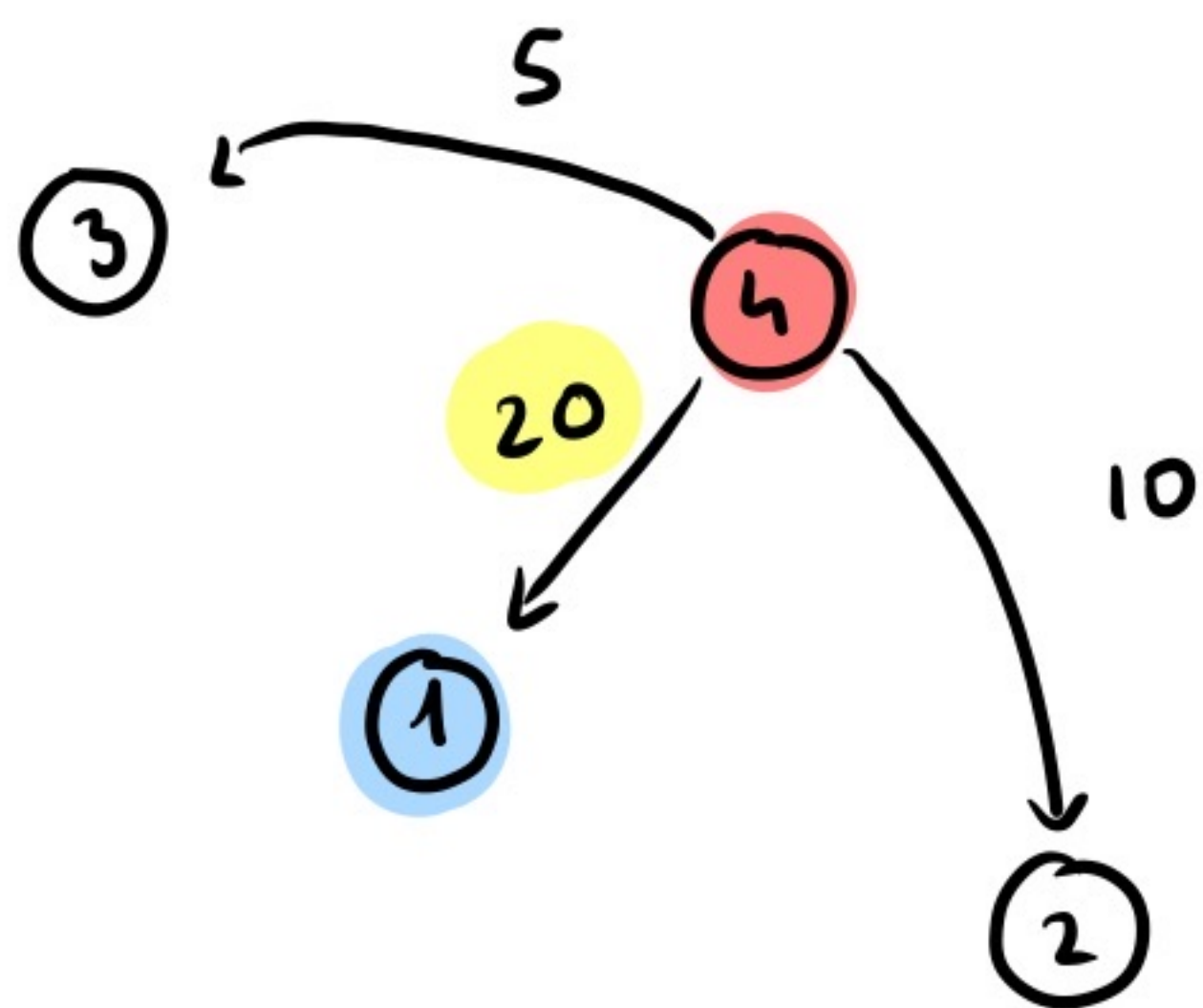
ex 2



+

Câți arbori de distanțe minime putem avea plecând din nodul 4?

#



Retinem 6 ca lista de adiacență

$$D[i] = \infty, \overline{1, n}$$

$$NR[i] = 0, \overline{1, n} \quad (\text{numarul de drumuri})$$

$$NR[S] = 1$$

$$PQ. \text{ push } (\{ D[S], S \})$$


```
while ( ! PQ.empty() )
```

```
{
```

```
    node = PQ.top().second
```

```
    if ( viz[ node ] == 1 )
```

```
        continue
```

```
    viz[ node ] = 1
```

```
    For x ∈ L[ node ]
```

```
        u = x.first
```

```
        w = x.second
```

```
        if D[u] > D[node] + w
```

```
            D[u] = D[node] + w
```

```
            NR[u] = NR[node]
```

```
            PQ.push ( D[u], u )
```

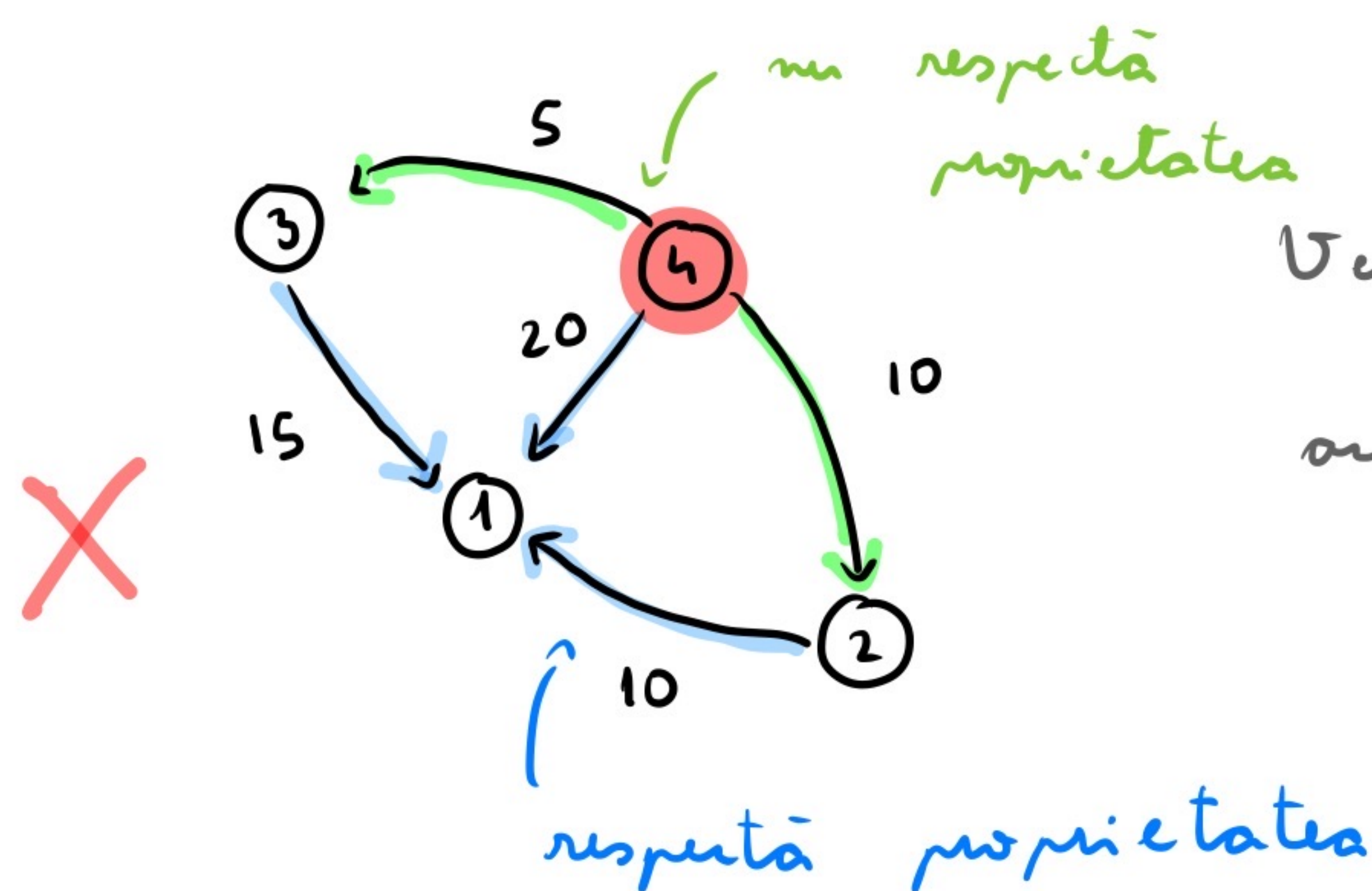
```
        else if D[u] == D[node] + w
```

```
            NR[u] += NR[node]
```

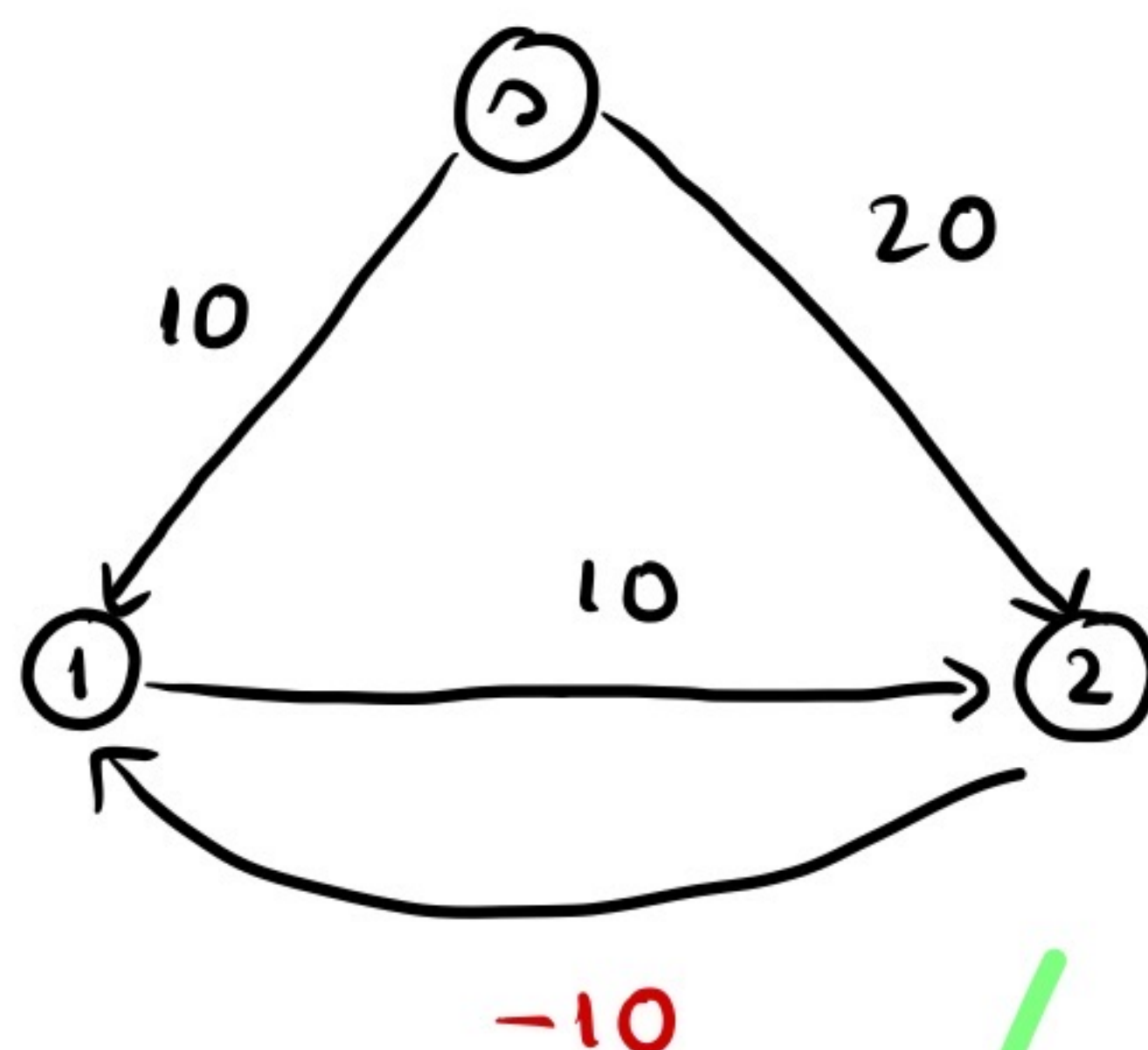
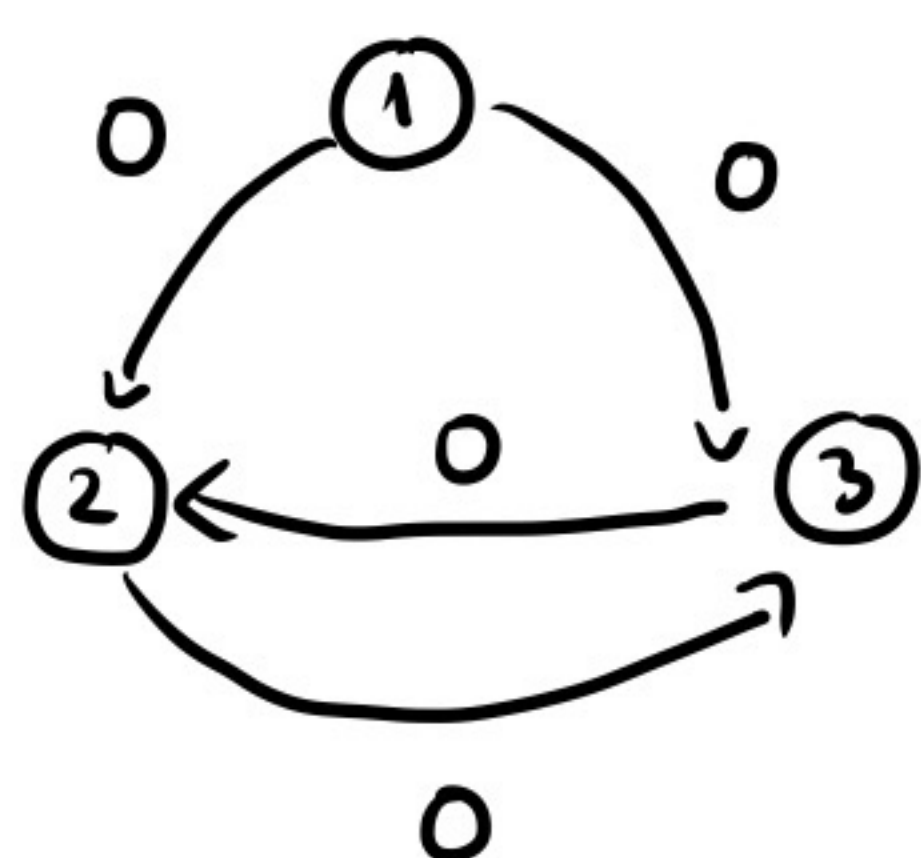
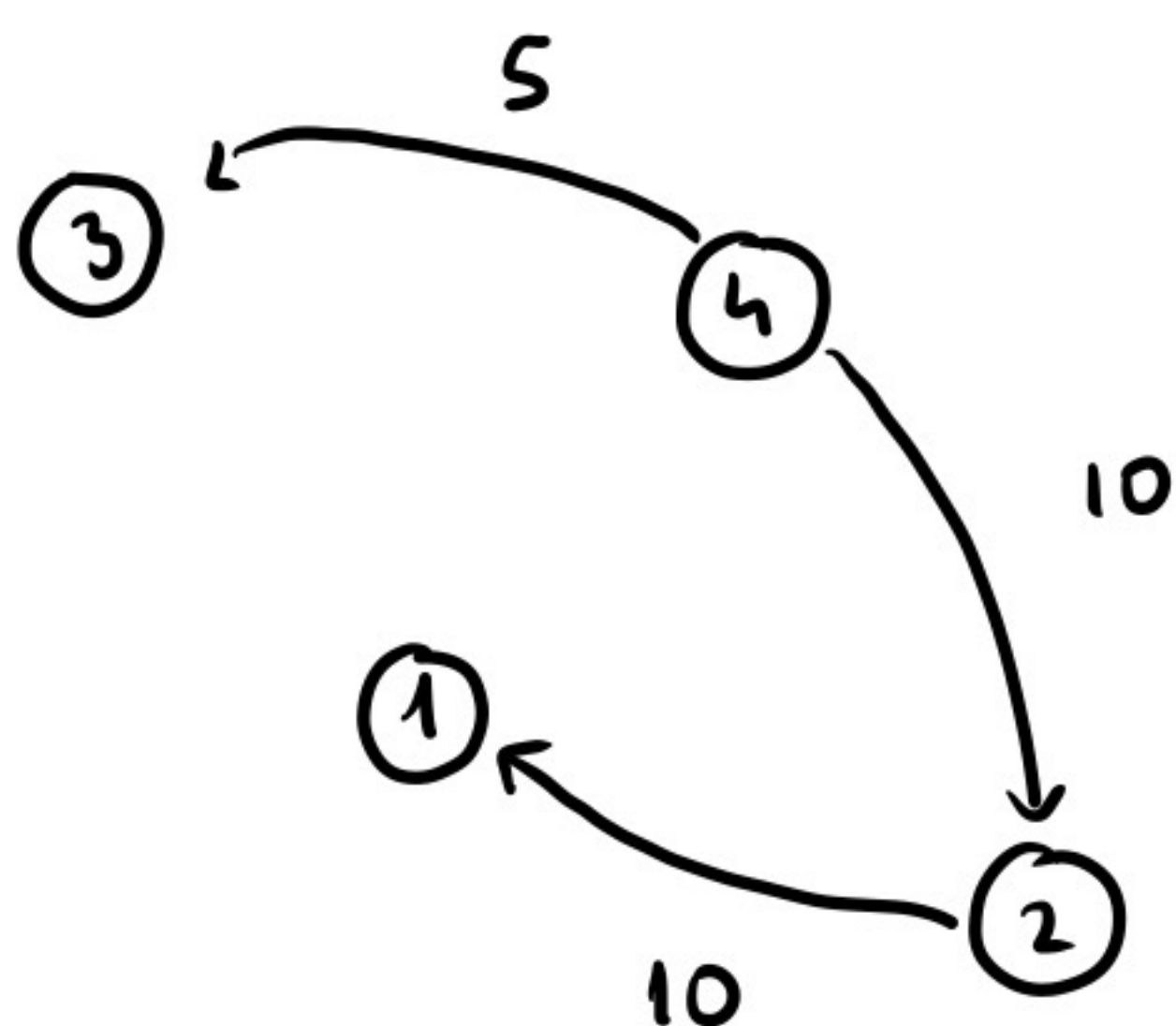
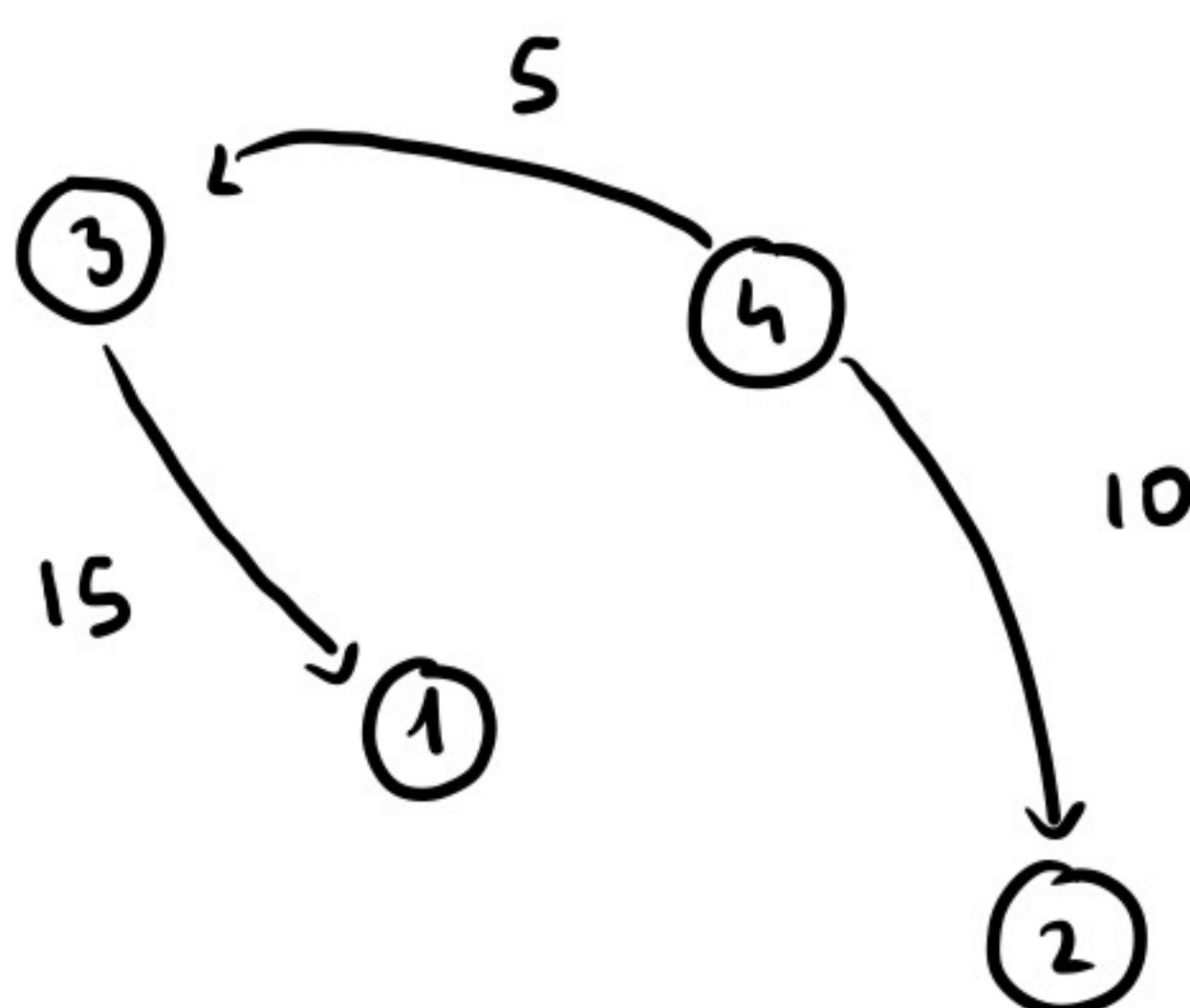
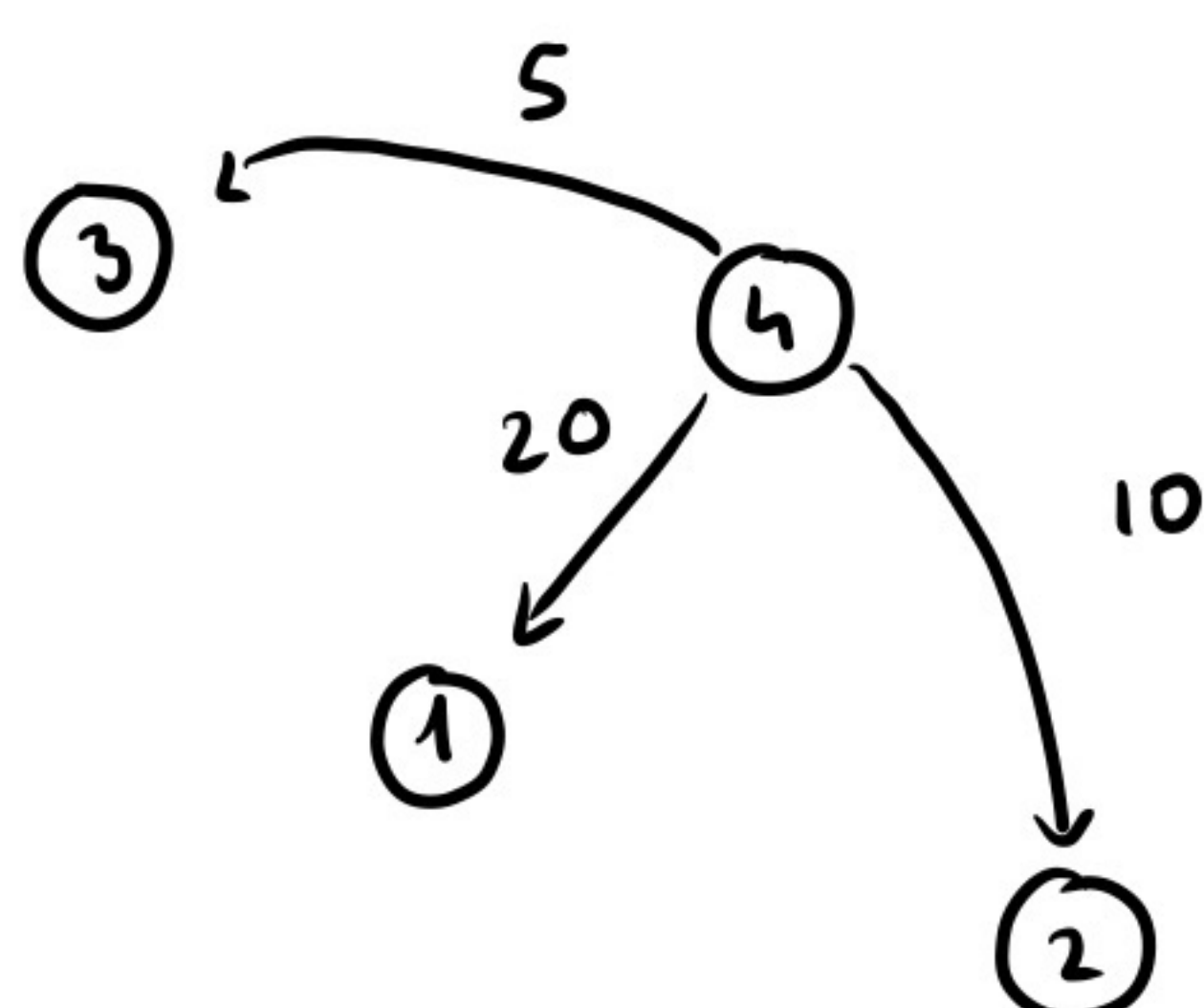
```
}
```


ex 3

Dați un exemplu de graf orientat pentru care
 $\exists$ un varf s a.i. pentru $\forall$ arc (u, v) $\exists$ un arbore
 de distanțe față de s care să conțină
 arcul (u, v) și unul care să nu îl conțină
 #



Verificăm proprietatea pentru
 acest graf



ex 4 problema 2 din coloana A7 1

DAG $\Rightarrow$ putem face sortare topologică

K noduri (nursă)

$O(n + m)$

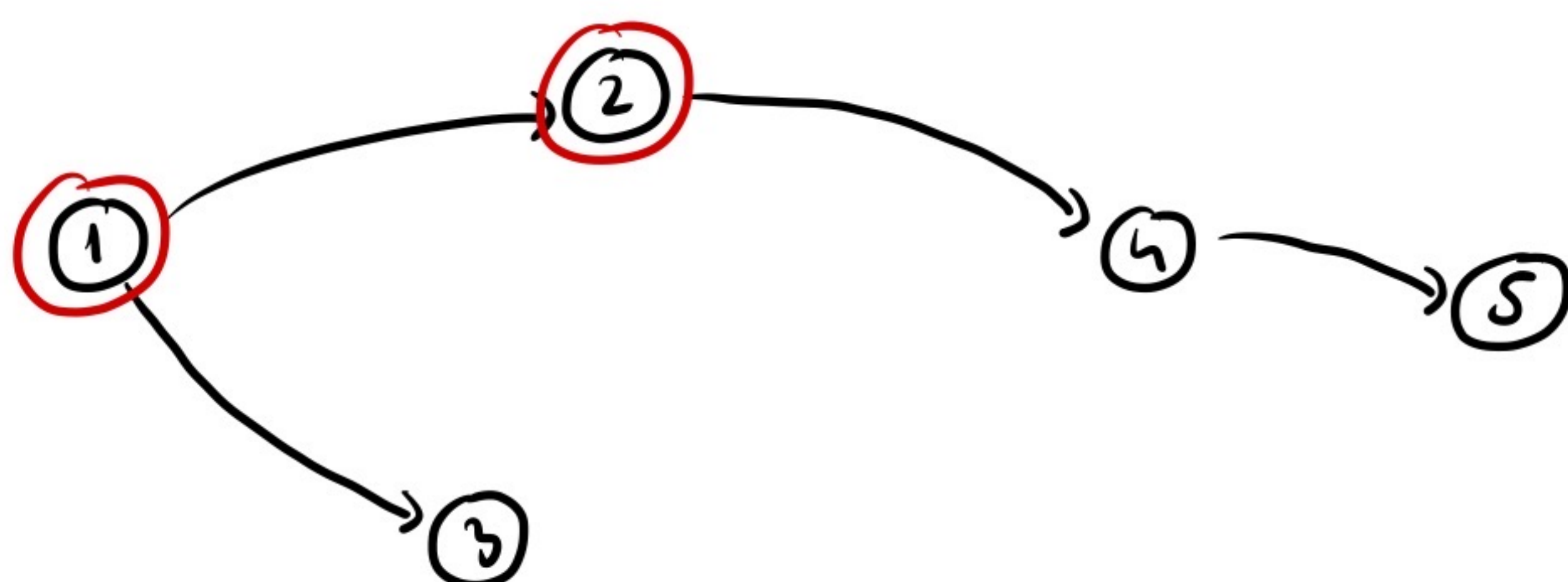
+

cel mai lung drum pornind dintr-un nod dat

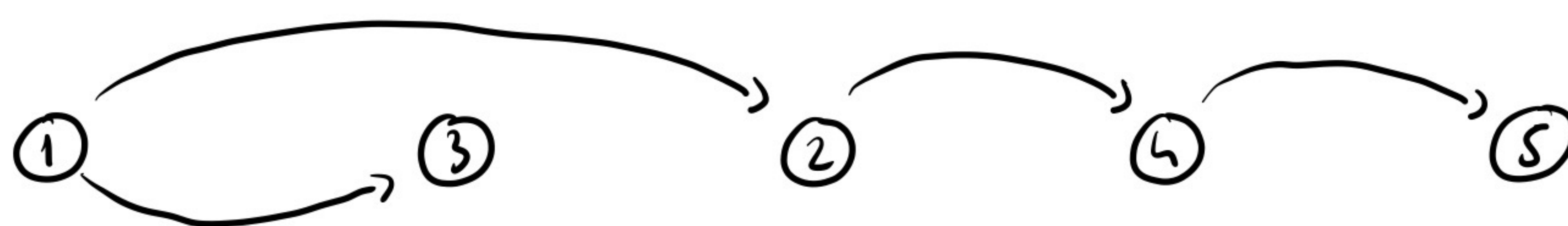
#

în alg. dorim punem
în cele K noduri $d[K] = 0$

DAG = graf orientat fără
cicluri



sortare
topologică



$D[i]$ = distanța maximă de la unul din
cele K noduri până la nodul i

1	2	3	4	5
0	0	$-\infty$	$-\infty$	$-\infty$
	1	1	2	3

cin >> n >> m >> k

for $i = \overline{1, k}$

cin >> v[i]

nodurile din care
putem incepe

for $i = \overline{1, n}$

cin >> x >> y

L[x].pb(y)

grad de intrare

grad[y]++

for $i = \overline{1, n}$

if grad[i] == 0

Q.pb(i)

while ! Q.empty()

nod = Q.front();

top = nod.pb(nod)

Q.pop();

For vein $\in L[nod]$

grad[vein]--

if grad[vein] == 0

Q.pb(vein)

$d[i] = -\infty, \quad i = \overline{1, n}$

For $i = \overline{1, K}$

$\Delta[v[i]] = 0$

For $x \in \text{top_root}$

For $vein \in L[x]$

if $\Delta[vein] < \Delta[x] + 1$

$\Delta[vein] = \Delta[x] + 1$

$p[vein] = x$

// afisarea drumului

For $i = \overline{1, n}$

if $\Delta[i] > \text{rol}$

$\text{rol} = \Delta[i]$

$\text{nod} = i$

$\text{cout} \ll \text{rol} \quad // \text{lungimea drumului}$

while $\text{nod} \neq 0$

$\text{drum} . \text{pb}(\text{nod})$

$\text{nod} = p[\text{nod}]$

} se iese în ordine inversă

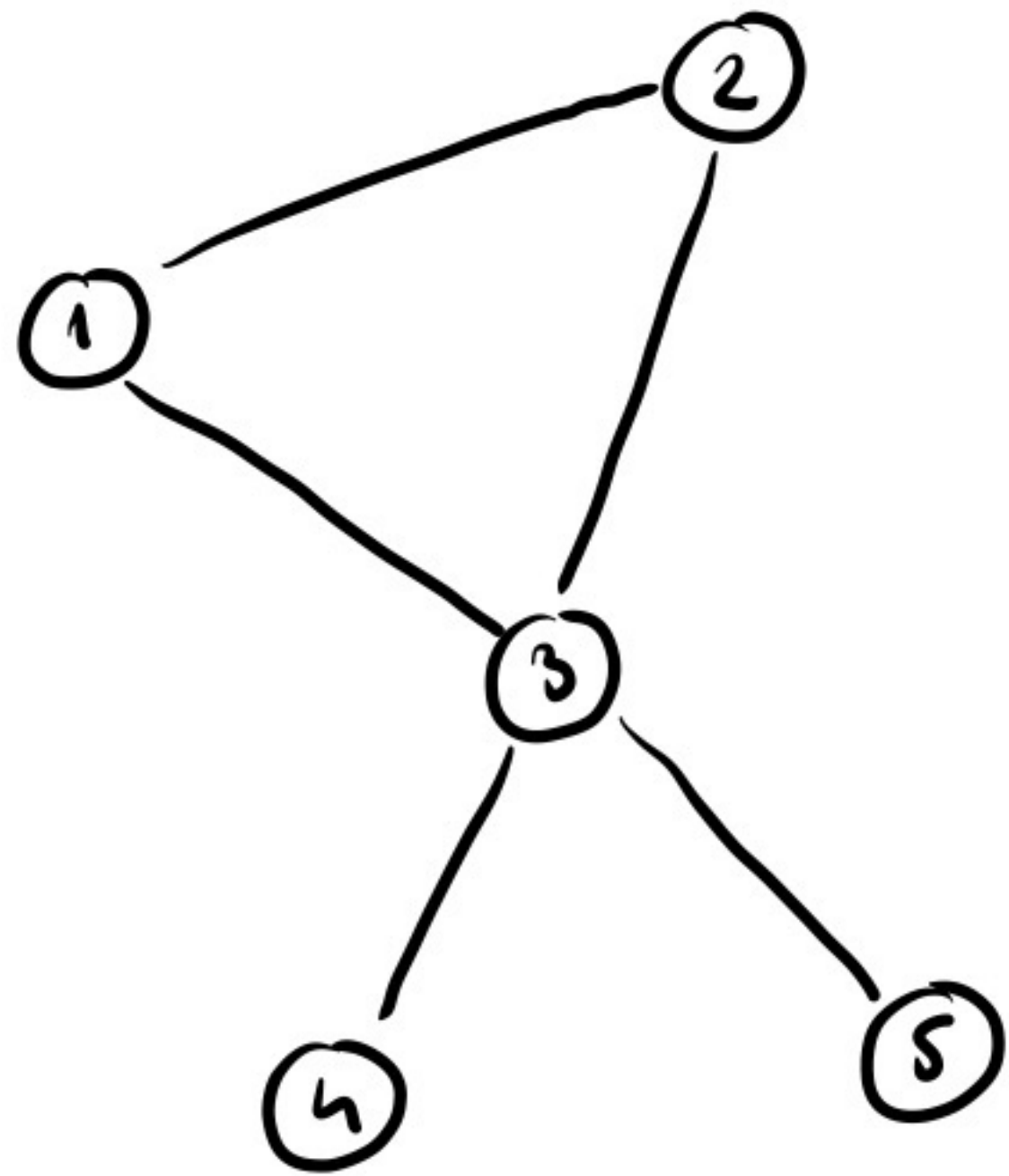
$\text{reverse}(\text{drum});$

$\text{print}(\text{drum});$

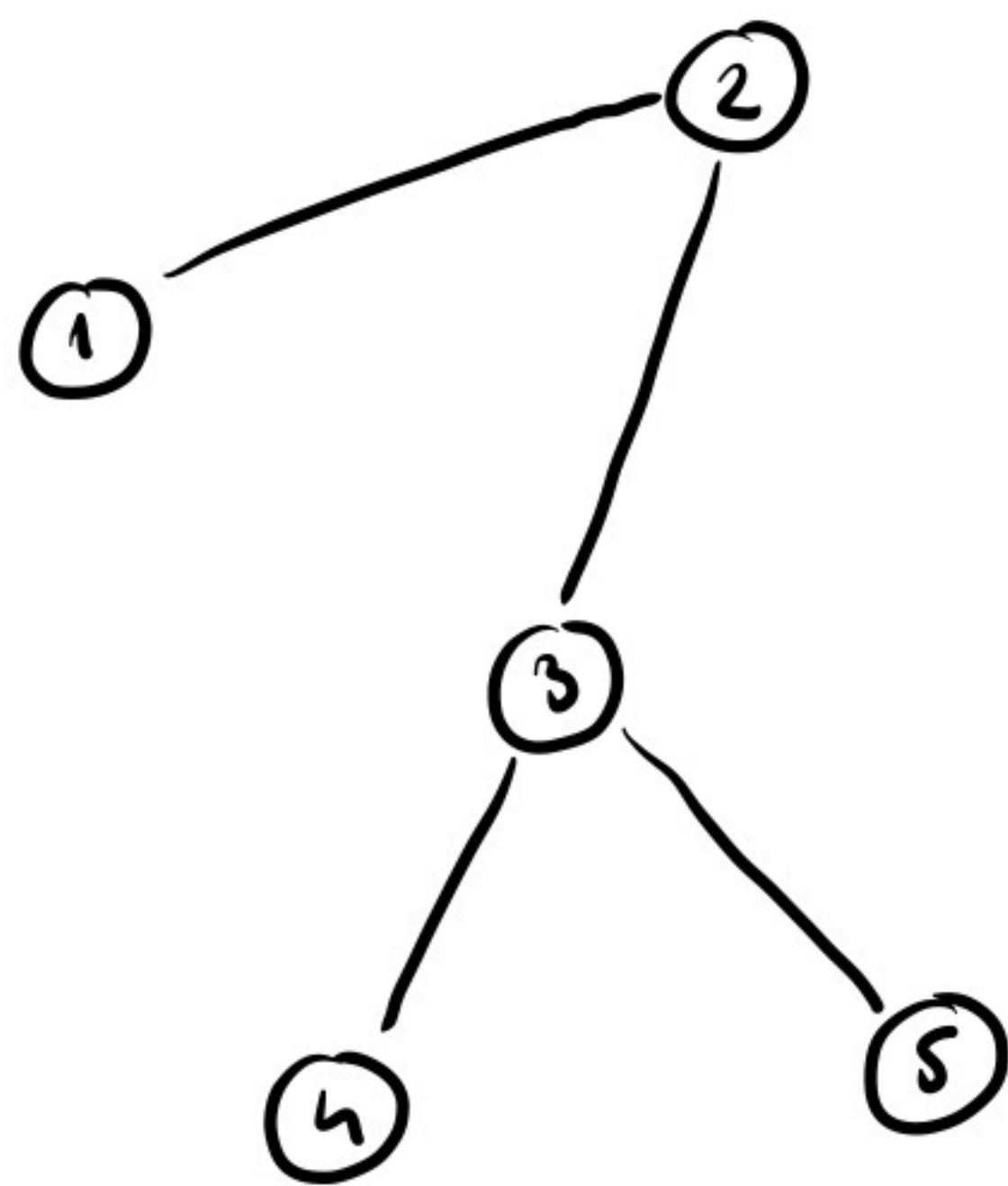
ex 5

G un graf neorientat fara costuri
+

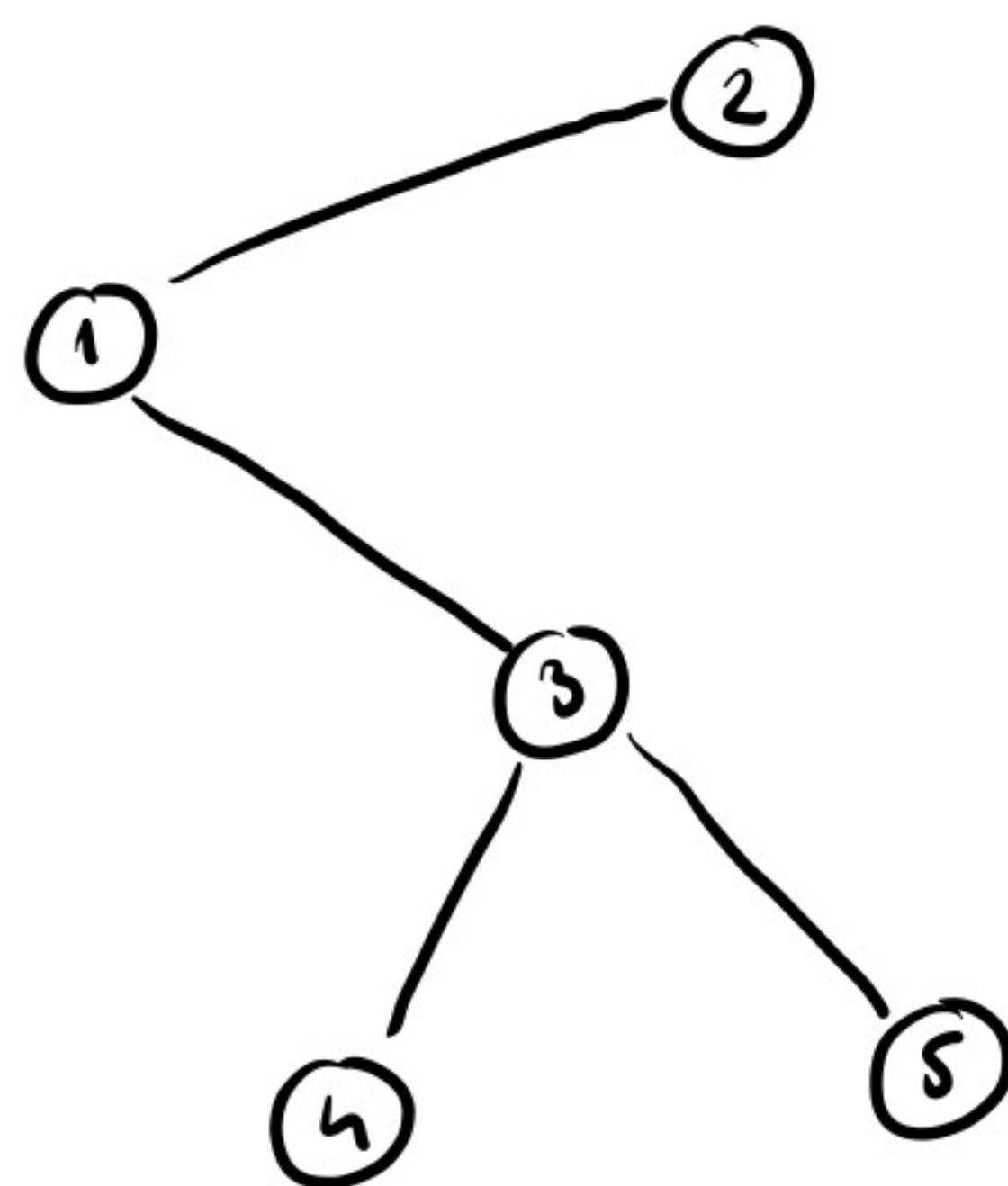
verif dacă G are cel puțin 2 arbori parțiali
#



AP1



AP2



1) G conex

$$1.1. \quad m > n - 1 \quad \Rightarrow \quad DA$$

$$1.2. \quad m \leq n \quad \Rightarrow \quad NU$$

2) G nu este conex $\Rightarrow NU$

Pt afișarea celor 2 arbori parțiali

$\Rightarrow$ Arborele DFS

$\Rightarrow$ Arborele BFS